

CS496 Blockchain Module

Exam-Ready Inline Diagram Study Guide: Blockchain, Bitcoin, Ethereum, Solidity, Solana, and Polkadot

Built from the supplied course archive. This version puts the important diagrams directly inside the study notes, so you can study the concepts and visuals together. It still covers every blockchain source file and includes code examples, exam notes, cheat sheet, questions, and focus topics.

Included source	Status
_021 Blockchain Overview.pptx - 26 slides	Covered; full slide snapshots included.
_022 Bitcoin Proof of Work Algorithm.pptx - 13 slides	Covered; code/formula/toy hash example included.
_023 How Bitcoin Works.pptx - 11 slides	Covered; slide 11 is blank but included in appendix.
_024 Ethereum How It Works -1.pptx - 32 slides	Covered; full node, transaction path, development view, Ethash included.
_025 Solidity By Example1.docx	Covered; all example categories included.
_026 Ethereum Smart Contracts Practice Sheet.docx	Covered; architecture diagram and practice examples included.
_027 Ethereum Proof of Stake.pptx - 19 slides	Covered; slide 19 is blank but included in appendix.
_028 Solana Proof of History.pptx - 11 slides	Covered; PoS/PoH/Tower BFT/upgrades included.
_029 Polkadot.pptx - 11 slides	Covered; architecture, DOT, NPoS, features included.

How to study this PDF

- 1 Study the chapter notes first. They are ordered to match the course material.
- 2 Use the yellow Exam-important boxes as memorization targets.
- 3 Use the inline diagrams beside each topic. The earlier full appendix PDF remains only as raw source backup; this file is the one to study.
- 4 Use the final cheat sheet and exam questions for fast revision.

1. Blockchain Overview

1.1 Origin of Bitcoin and blockchain

The blockchain module begins with Satoshi Nakamoto's 2008 paper, "Bitcoin: A Peer-to-Peer Electronic Cash System." Bitcoin became operational in 2009. The individual building blocks were not all new; the innovation was combining peer-to-peer networking, cryptography, a decentralized distributed ledger, proof-of-work, and economic incentives into one working system.

The main motivation is direct online payment without a financial institution. Digital signatures authorize payments, while a peer-to-peer timestamp/proof-of-work chain prevents double spending without relying on a central clearing party.

Exam-important: Bitcoin is not just "cryptography." It is cryptography + P2P networking + proof-of-work + distributed ledger + consensus rules.

1.2 Peer-to-peer network

A peer-to-peer network is a network of independent computers that share data without a central server. Each peer can provide services and consume services. In blockchain, this supports decentralization because nodes can independently store, verify, and propagate transactions and blocks.

1.3 Bitcoin block structure

A Bitcoin block has a general structure plus a fixed 80-byte header. The header is the part miners hash during proof-of-work.

Block component	Purpose	Size
Magic number	Identifies the Bitcoin network; slide shows 0xD9B4BEF9.	4 bytes
Block size	Number of bytes up to the end of the block.	4 bytes
Block header	Metadata for linking, mining, and validation.	80 bytes
Transaction counter	Number of transactions in the block.	1-9 bytes
Transactions	Actual transaction records.	Variable
Header field	Purpose	Size
Version	Block version/rules.	4 bytes
Previous block hash	Links this block to the previous block.	32 bytes
Merkle root	Hash summary of all transactions in the block.	32 bytes
Timestamp	Approximate block time.	4 bytes
Difficulty bits	Compact form of the proof-of-work target.	4 bytes
Nonce	Value miners change while searching for a valid hash.	4 bytes

Exam-important: previous block hash creates the chain; Merkle root protects transactions; nonce is changed during mining.

Block Structure in Bitcoin

Field	Description	Size
Magic no	The value is always 0xD9B4BEF9	4 bytes
Block Size	Number of bytes up to the end of the block	4 bytes
Block Header	Consists of six items	80 bytes
Transaction Counter	Integer count	1 - 9 bytes
Transactions	List of transactions	Variable

Slide image: Bitcoin block structure.



Field	Purpose	Size
Version	Block version	4 bytes
hashPrevBlock	256-bit hash of the previous block header	32 bytes
hashMerkleRoot	256-bit hash based on all of the transactions in the block	32 bytes
Time	Current timestamp in seconds since 1970-01-01T00:00 UTC	4 bytes
Difficulty Bits	Current target in a compact format (https://en.bitcoin.it/wiki/Target)	4 bytes
Nonce	32-bit number (starts at 0)	4 bytes

Slide image: Bitcoin block header.

1.4 Hash fields and tamper evidence

The previous block hash links blocks. The transaction hash/Merkle root summarizes all transactions. If any transaction changes, its hash changes, the Merkle root changes, and the block header hash changes. This breaks the chain unless proof-of-work is redone.

1.5 Digital signatures and account identity

Blockchain systems use public-key cryptography. A private key signs a transaction; the public key verifies it. Account addresses are derived from public-key material. The private key must remain secret because it controls the ability to authorize spending.

1.6 Double spending

Double spending is the attempt to spend the same digital money twice. Traditional finance solves this with banks, clearing systems, and guarantees. Bitcoin solves it by making the transaction history public, verified, timestamped, and ordered through proof-of-work consensus.

1.7 Transactions

The slides call blockchain systems distributed transactional systems. The transaction is the central entity: it is the object signed by users, propagated through the network, verified by nodes, and eventually included in blocks.

1.8 Hash-based proof-of-work

Proof-of-work is the consensus protocol used in Bitcoin-like systems. Nodes compete to solve a hard computational puzzle: computing a block header hash that satisfies a target format. The work is expensive to perform but easy for other nodes to verify.

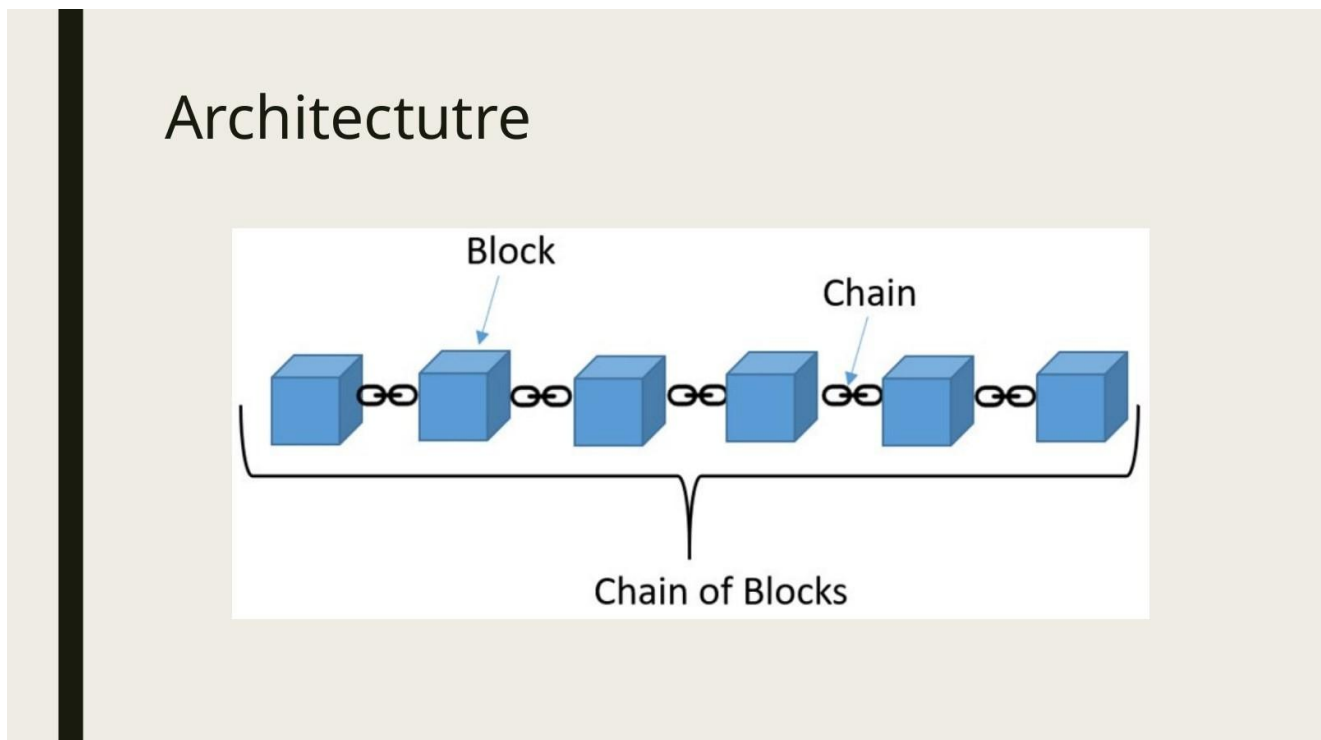
The slide says the hard problem is NP complete. For exam purposes, do not overstate this. Bitcoin mining is a brute-force hash search puzzle: hard because it requires many trials, not because nodes solve a traditional NP-complete optimization problem.

1.9 Definition of blockchain

A blockchain is a peer-to-peer distributed ledger that is cryptographically secure, append-only, extremely hard to change, and updateable only by consensus among peers. The term can refer to the ledger, network, consensus mechanism, or the entire blockchain technology system.

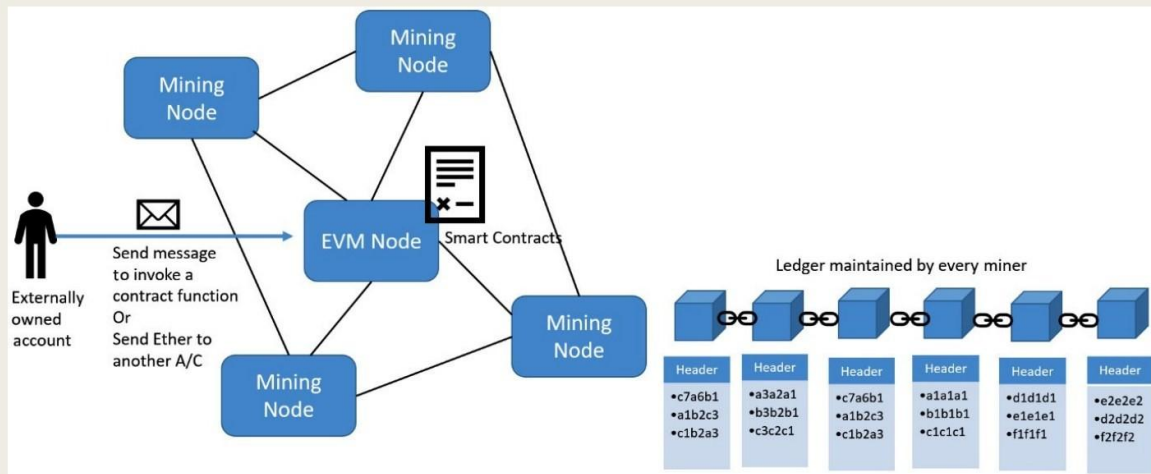
1.10 Blockchain structure and architecture

The architecture diagrams show a chain of blocks, mining nodes, external accounts/users, transactions/messages, smart contracts, an execution layer such as the EVM, and a ledger. The relationship is: users send transactions; nodes verify and propagate them; miners/validators create blocks; accepted blocks update the ledger.



Slide image: blockchain as a chain of blocks.

Architecture

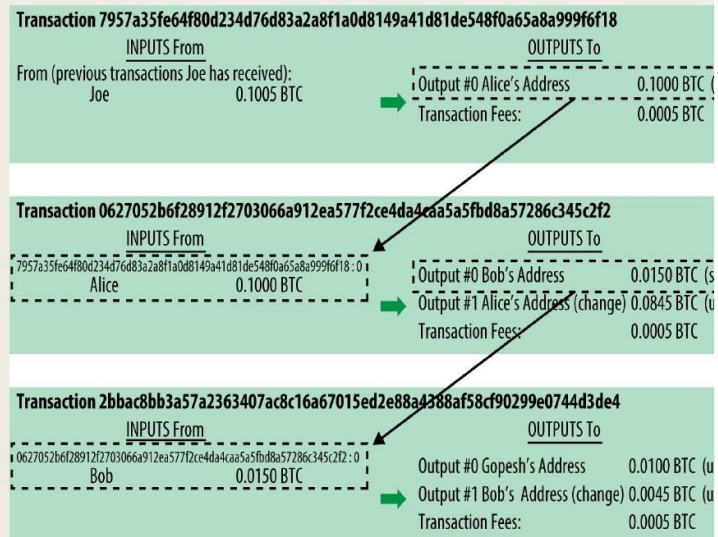


Slide image: relationship between blockchain and mining.

1.11 Transaction lifecycle

- 1 User/sender generates a transaction using wallet software or another interface.
- 2 Wallet signs the transaction using the sender's private key.
- 3 Transaction is broadcast to the blockchain network using flooding/gossip.
- 4 Nodes verify the transaction and propagate it further.
- 5 Mining nodes include transactions in candidate blocks.
- 6 Mining starts once the previous block has been broadcast by the winning node.
- 7 Miners compete to solve proof-of-work for their candidate block.
- 8 The first successful miner broadcasts the confirmed block.
- 9 Other nodes stop mining the old candidate and verify the winning block.
- 10 Transactions in the valid block are added to the ledger.

Some Example Transactions from Bitcoin



Important diagram: transaction generation, signing, and broadcast through the P2P network.

How to read it: wallet creates/signs transaction -> transaction is broadcast/flooded -> nodes verify and relay it -> mining/validator nodes eventually include it in a block.

Block Structure

Size	Field	Description
4 bytes	Block Size	The size of the block bytes, following this field
80 bytes	Block Header	Several fields form the block header
1-9 bytes (VarInt)	Transaction Counter	How many transactions follow
Variable	Transactions	The transactions recorded in this block

Important diagram: mining nodes assemble transactions and begin mining candidate blocks.

Exam target: be able to explain why many miners can build candidate blocks at the same time, but only the first valid accepted block extends the chain.

1.12 Mining process and emergent consensus

In Bitcoin, mining is expected roughly every 10 minutes. Transactions generated during this period are packed into blocks by mining nodes. Each node builds its own candidate block, but only the winning block is propagated and accepted.

Consensus emerges from four processes: independent transaction verification by full nodes; independent transaction aggregation into blocks by miners with proof-of-work; independent block verification and chain assembly by every node; and acceptance of the chain with the most cumulative demonstrated computation.

Exam-important: Bitcoin consensus is emergent. No central server announces the correct chain. Nodes independently follow the same rules, which leads them to converge on the chain with the most cumulative proof-of-work.

1.13 Features: decentralized, distributed, database

Feature	Meaning in the slides
Decentralization	The service stays available even if a server or group of servers fails; no single server has absolute control over data/execution.
Distributed	Nodes are connected many-to-many, not one-to-one or one-to-many.
Database	Stores durable data that can be accessed later and managed through retrieval/backup/import/export-like functionality.

2. Bitcoin Proof-of-Work Algorithm

2.1 Meaning of proof-of-work

Proof-of-work is evidence that a miner spent computational effort. In Bitcoin, the miner searches for a block header hash that is numerically less than a target.

2.2 Target and difficulty

The challenge is to produce a hash below a pre-specified target. A higher target means easier mining; a lower target means harder mining. Therefore, target and difficulty are inversely related.

```
def proof_of_work(header, difficulty_bits):
    target = 2 ** (256 - difficulty_bits)
    for nonce in xrange(max_nonce):
        hash_result = hashlib.sha256(str(header)+str(nonce)).hexdigest()
        if long(hash_result, 16) < target:
            return (hash_result, nonce)
```

Exam-important: lower target = higher difficulty. The miner succeeds only when hash_result < target.

2.3 What gets hashed?

The string hashed by the miner is the block header plus a nonce. Nonce means "number once." In practice, miners repeatedly change the nonce and other mutable block data until the hash satisfies the target.

```
hash_result = hashlib.sha256(str(header) + str(nonce)).hexdigest()
```

The Target

- As can be seen from the above code, the target is computed using "difficulty bits"

$\text{target} = 2^{256 - \text{difficulty_bits}}$

These are usually leading bits in the resulting hash

Important diagram: target calculation from difficulty bits.

How to read it: $\text{target} = 2^{(256 - \text{difficulty_bits})}$. A valid block hash must be numerically below this target. Increasing difficulty_bits lowers the target and makes success rarer.

2.4 Toy SHA-256 run

The slides show a toy example where the text "I am Mohammed ELAffendi" is concatenated with nonce values 0 through 19. Each nonce produces a completely different SHA-256 hash. The point is unpredictability: miners cannot know the winning nonce in advance; they must trial many values.

```
text = "I am Mohammed ELAffendi"
for nonce in xrange(20):
    input = text + str(nonce)
    hash = hashlib.sha256(input).hexdigest()
    print input, '=>', hash
```

```

0 => dac6fe1eac2cf9e82dd1e6f71559a6f703c7d3fe42aaa442e8d4741be6d9c6e1
1 =>
8e500e40719f0a7146b0077563fddb3d8d2d6d52c1ea3c0d31597a29ce97a8e
2 => 34be29ccb2160c67aab0bf89e165a0528054de15fd263f120c72f84cd6467421
3 =>
9b86f88367143dc6c2a15daa52116948754dc35b8638e72e22d423e1e5994c5d
4 => 2c2979a20142c994e65dc764e10f1a9dc8eafbd8444de1687ddc80f9be2c8127
5 => 9ae4c1eb30467eab845cf2c6bfc4ef30c42a0958691a264af1b8188a2f265855
6 => b9708e2725f323ea6f53fb3ad5ed0424a52d6b931d96804a22d8672e90e7af75
7 =>
3a9765b015fe8016906e3c83e53cbe4d866473fa4ca02018a15317496b791e94
8 => 317d2e0b38ee2d2e39847734bc5643cf7066cb40d8f03a52700aa26ad7f5d016
9 =>
6b3c9be7bcda7c8c7e9d916ef05eb4ef03e7d8d75df94de1d3b18a5326d864a4
10 =>
3b47698a1ed5d12ac2ae5eea4a9d2a7ff2220de66138d6f2fa8384873e0e8510
11 =>
59c922d29e6db10cfb2194e2ab8e3b2db3aeb658941356bf794eddba67642c30
12 =>
4745ba3db7597f450ff253df6c3325ef1e8d037867d8429ed9a6653b16b39310
13 => fd1283170680b60de665a73aad7c9cc1cd6ca9c0f58ce89c4c6a07fef2052420
14 => 550cf003ea24915e05b874cf27d91b114f5f665a785cdee45fdc648e4c0c1cff
15 =>

```

Important diagram: toy SHA-256 nonce output table.

How to read it: the same text with nonce 0, 1, 2, ... gives completely different hashes. This is why mining is trial-and-error, not direct calculation.

2.5 Leading zero bits and time doubling

The slides test difficulty from 0 to 31 bits. Difficulty is explained as leading bits/leading-zero-style constraints in the resulting hash. Each added difficulty bit roughly halves the acceptable hash space, so expected computation time roughly doubles.

Exam-important: increasing difficulty by one bit approximately doubles the computational work.

2.6 Difficulty adjustment

Proof-of-work has a timing role: it keeps Bitcoin's heartbeat near 10 minutes per block. Every 2,016 blocks, nodes retarget difficulty by comparing actual time against expected time.

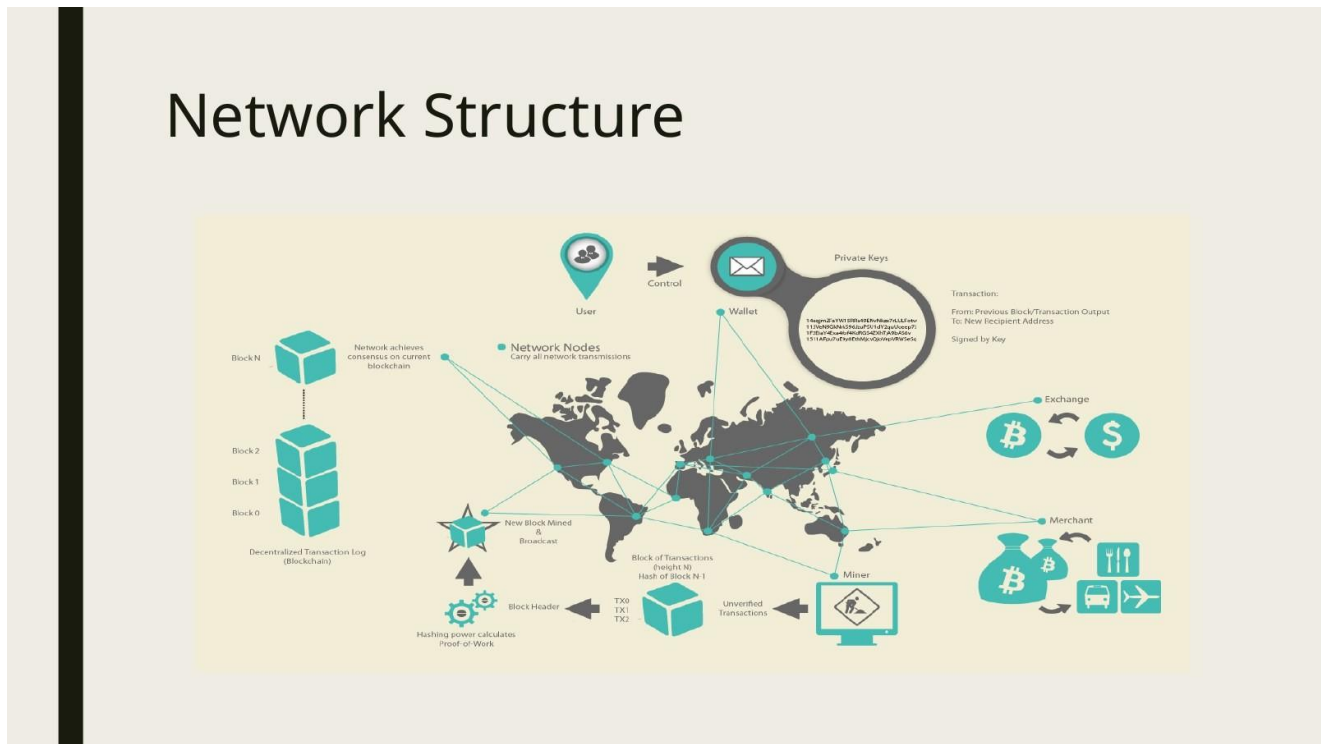
$2,016 \text{ blocks} * 10 \text{ minutes} = 20,160 \text{ minutes}$

If blocks were found faster than every 10 minutes on average, difficulty increases and target decreases. If blocks were found slower than expected, difficulty decreases and target increases.

3. How Bitcoin Works: A Transaction Journey

3.1 Network structure and coffee example

The Bitcoin transaction journey starts from a user-facing payment situation, such as buying coffee. The customer can pay in fiat dollars or in BTC/satoshis. The important point is that the wallet hides most transaction construction details from the user.



Slide image: Bitcoin network overview.

3.2 What a transaction means

A transaction tells the network that the owner of some bitcoin value has authorized transferring that value to another owner. The new owner can later create another transaction, forming a chain of ownership.

3.3 Inputs, outputs, change, and fee

Bitcoin uses the UTXO model: unspent transaction outputs are consumed as inputs and new outputs are created. A transaction often has a change output back to the sender. The fee is the difference between total inputs and total outputs.

Part	Meaning
Inputs	Previous unspent outputs being spent.
Outputs	New spendable outputs created by this transaction.
Change output	Leftover value returned to the sender.
Fee	Total inputs minus total outputs. Paid to the miner/validator in Bitcoin mining context.

Slide example: inputs of 0.10 + 0.20 + 0.10 + 0.15 BTC total 0.55 BTC; outputs of 0.10 + 0.20 + 0.20 total 0.50 BTC; difference is a 0.05 BTC fee.

Typical Transaction

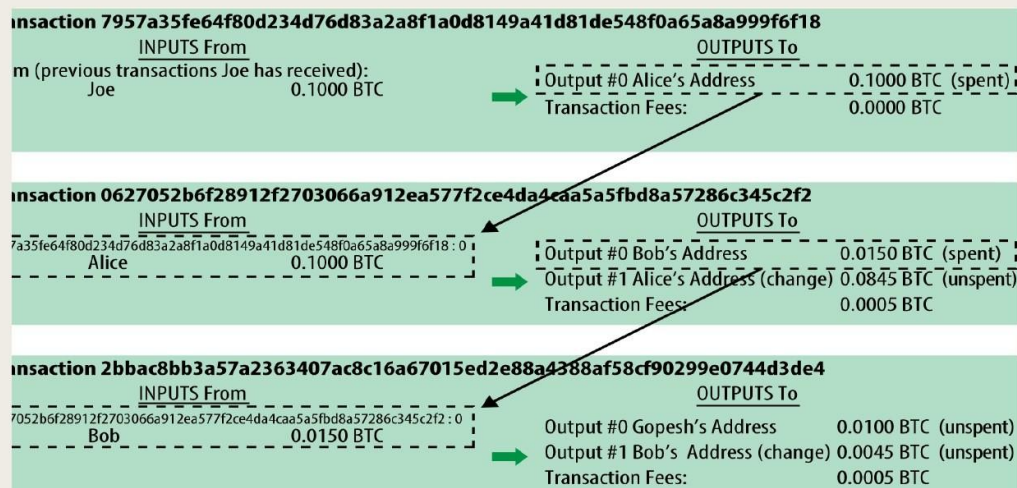
Transaction as Double-Entry Bookkeeping			
Inputs	Value	Outputs	Value
Input 1	0.10 BTC	Output 1	0.10 BTC
Input 2	0.20 BTC	Output 2	0.20 BTC
Input 3	0.10 BTC	Output 3	0.20 BTC
Input 4	0.15 BTC		
Total Inputs:	0.55 BTC	Total Outputs:	0.50 BTC
-	<i>Inputs</i>		<i>0.55 BTC</i>
	<u><i>Outputs</i></u>		<u><i>0.50 BTC</i></u>
	<i>Difference</i>		<i>0.05 BTC (implied transaction fee)</i>

Slide image: typical Bitcoin transaction.

3.4 Chain of transactions and common payment pattern

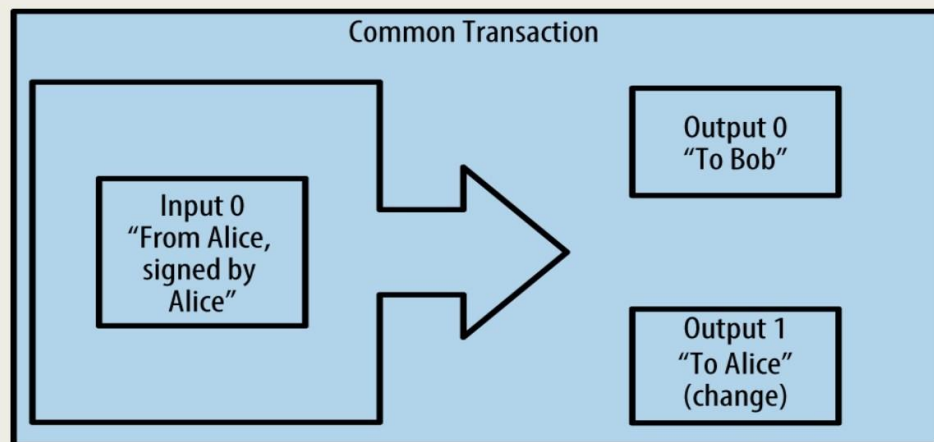
The transaction chain diagram shows that outputs from earlier transactions become inputs to later transactions. A common payment has one or more inputs from the sender, one output to the recipient, and one change output back to the sender.

Chains of Transactions



Slide image: chain of transactions.

How Payments are Performed



Slide image: common transaction with recipient output and change output.

3.5 Wallet application

The wallet application contains the logic for choosing inputs, creating outputs, calculating change, signing the transaction, and preparing it for broadcast. The user normally specifies only destination and amount. The wallet can construct transactions even while offline, then broadcast later when connected.

Exam-important: Bitcoin wallets do not literally store coins. They manage keys and construct transactions that spend UTXOs.

3.6 Real transaction and mining

The real transaction screenshot demonstrates actual inputs, outputs, spent/unspent status, fees, and confirmations. The mining diagram also introduces block height and block depth. More depth means more confirmations and a lower chance of reversal.

A Real Transaction

Transaction View information about a bitcoin transaction

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

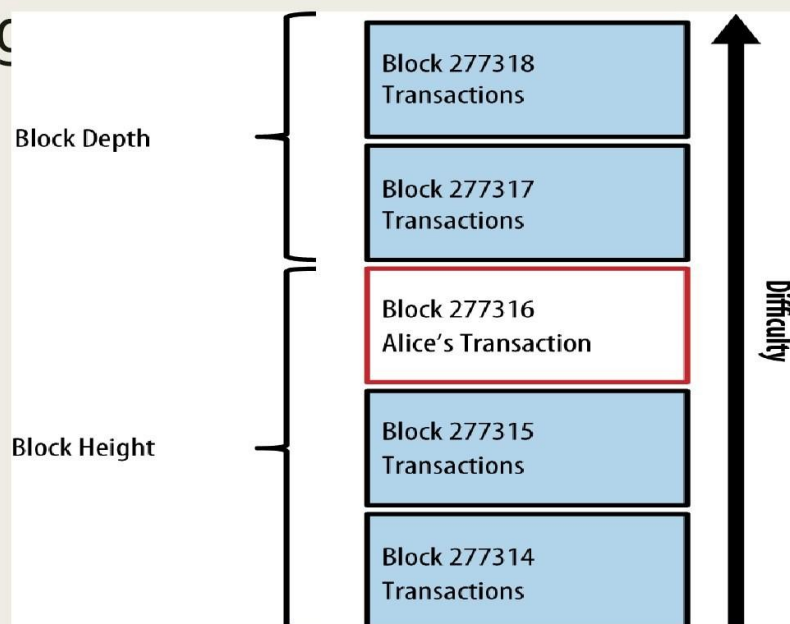
1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK (0.1 BTC - Output) → 1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA - (Unspent) 0.015 BTC
1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK - (Unspent) 0.0845 BTC

97 Confirmations 0.0995 BTC

Summary		Inputs and Outputs	
Size	258 (bytes)	Total Input	0.1 BTC
Received Time	2013-12-27 23:03:05	Total Output	0.0995 BTC
Included In Blocks	277316 (2013-12-27 23:11:54 +9 minutes)	Fees	0.0005 BTC
		Estimated BTC Transacted	0.015 BTC

Slide image: real Bitcoin transaction example.

Mining

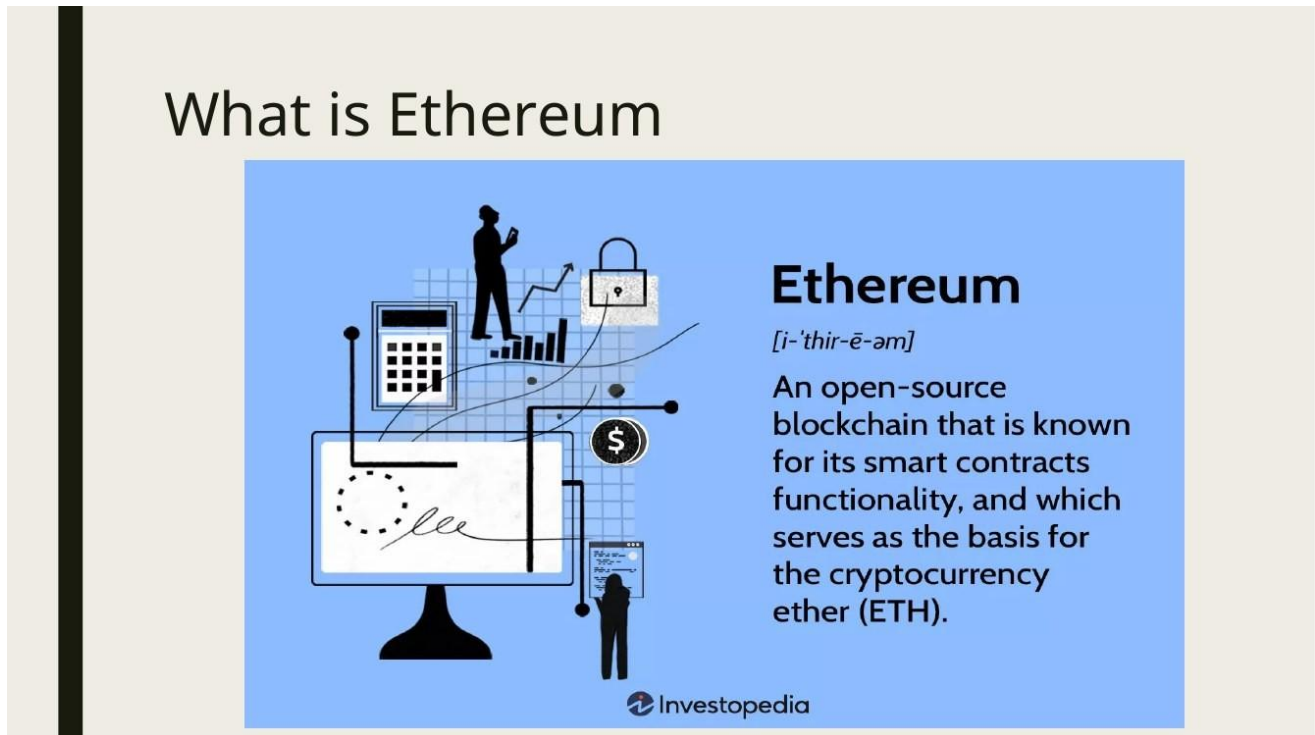


Slide image: Alice's transaction included in a mined block.

4. Ethereum: How It Works

4.1 What Ethereum is

Ethereum is a global blockchain-based software platform powered by its native cryptocurrency, ether (ETH). Its core difference from Bitcoin is programmability: Ethereum supports smart contracts and decentralized applications, not only value transfer. The slides describe it as scalable, programmable, secure, and decentralized.



Slide image: Ethereum overview visual.

4.2 History

Vitalik Buterin published the Ethereum white paper in 2014. Ethereum launched in 2015 with contributors including Buterin and Joe Lubin. Its founders emphasized blockchain potential beyond secure virtual payments.

4.3 Ethereum accounts

Account type	Controlled by	Can initiate transactions?	Has code?
Externally Owned Account (EOA)	Private key	Yes	No
Contract Account	Contract code	No; reacts when called	Yes

EOAs can send value to other EOAs or call contract accounts. A message from an EOA to a contract account activates the contract code. Contract accounts cannot initiate new transactions independently.

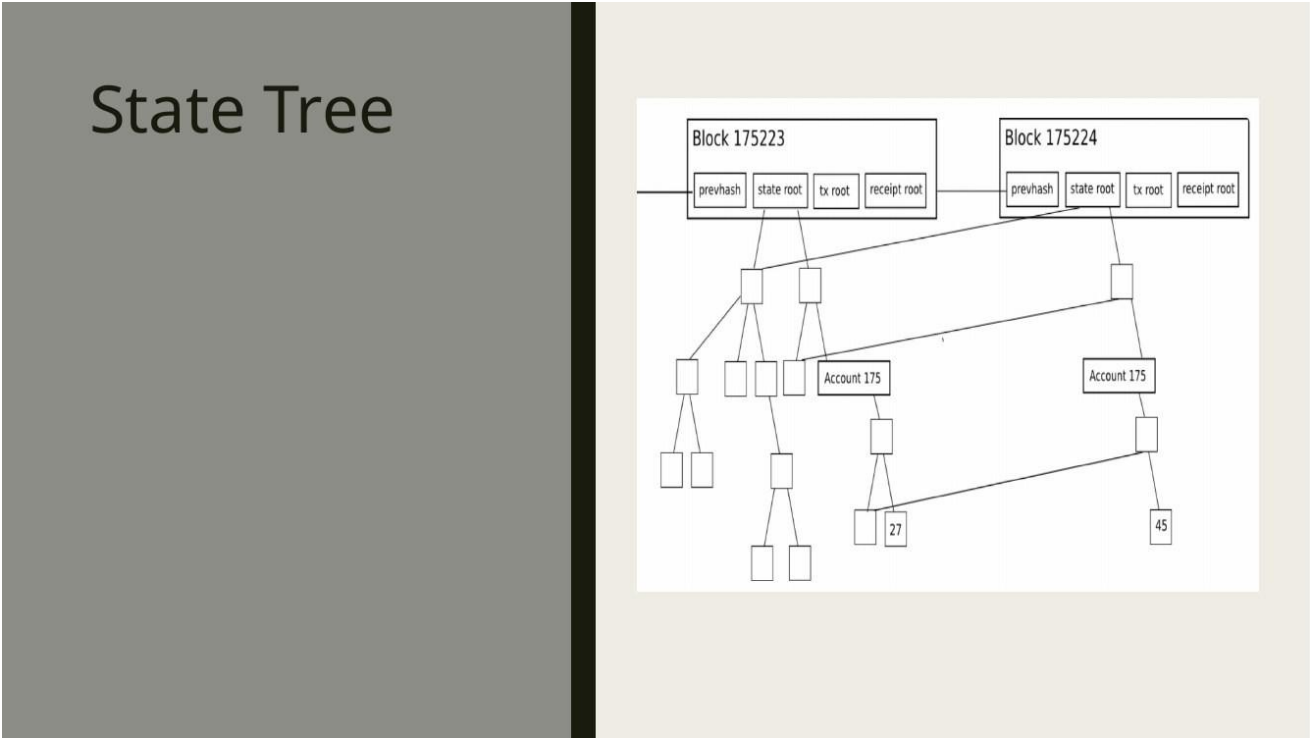
Exam-important: EOAs initiate transactions; contract accounts execute code only when triggered.

4.4 Account state

State field	Meaning
nonce	For EOAs: number of transactions sent. For contract accounts: number of contracts created by the account.
balance	Amount of Wei owned by the address.
storageRoot	Root hash of the Merkle tree encoding this account's storage.
codeHash	Hash of EVM bytecode for contract accounts; hash of empty string for EOAs.

4.5 State tree

Ethereum state is a key-value map: address -> account declaration. Each account declaration includes balance, nonce, code, and storage. The storage itself is also represented as a tree.

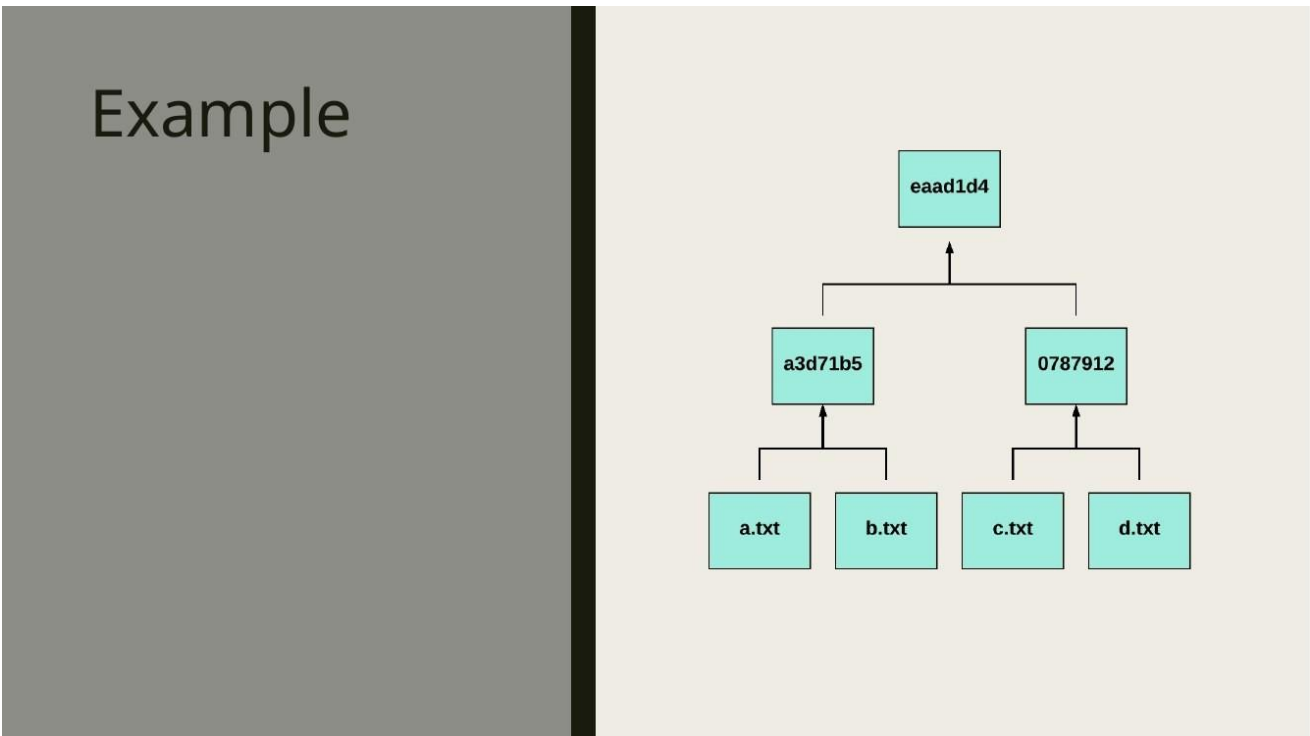


Slide image: Ethereum state tree.

4.6 Merkle trees

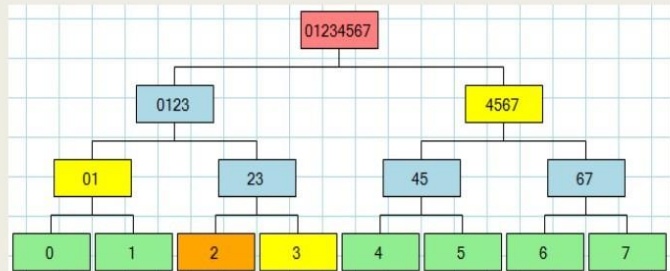
A Merkle tree hashes chunks of data into buckets, hashes the buckets, and repeats until only one root hash remains. A binary Merkle tree combines two inputs into one output at each internal node.

Purpose: efficient tamper detection and inclusion proofs. To prove that a value is included, a verifier needs only the value and the hashes along the path to the root, not the entire dataset.



Slide image: Merkle tree example.

Validation:
Prove that
"2" is
included



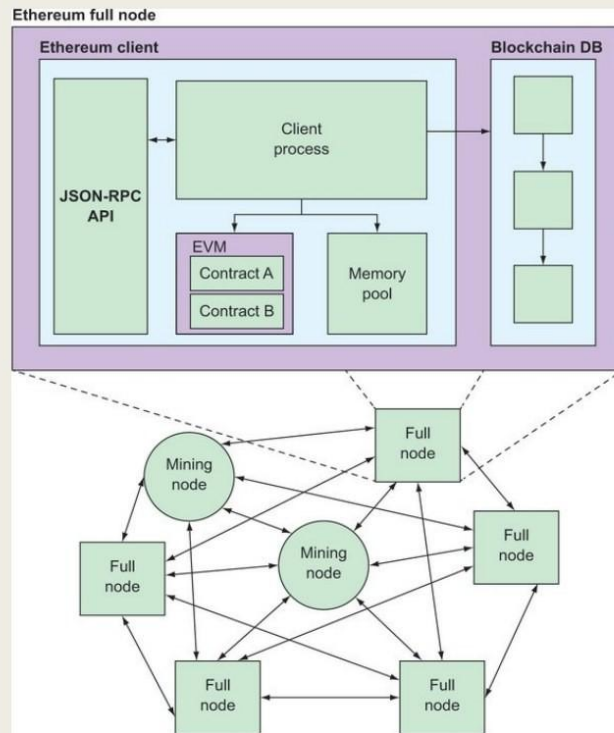
Slide image: proving that value 2 is included.

4.7 Inside an Ethereum full node

An Ethereum node includes an Ethereum client and blockchain database. The client contains the EVM, memory pool, client process, and JSON-RPC API.

Component	Role
EVM	Executes smart contract bytecode, typically compiled from Solidity.
Memory pool	Stores received transactions before propagation/inclusion.
Client process	Coordinates messages, transactions, EVM dispatch, mempool, and incoming blocks.
JSON-RPC API	Exposes node functionality to wallets, DApps, users, and other programs.
Blockchain database	Stores blocks, transactions, deployed contract bytecode, and state.

The Full Node

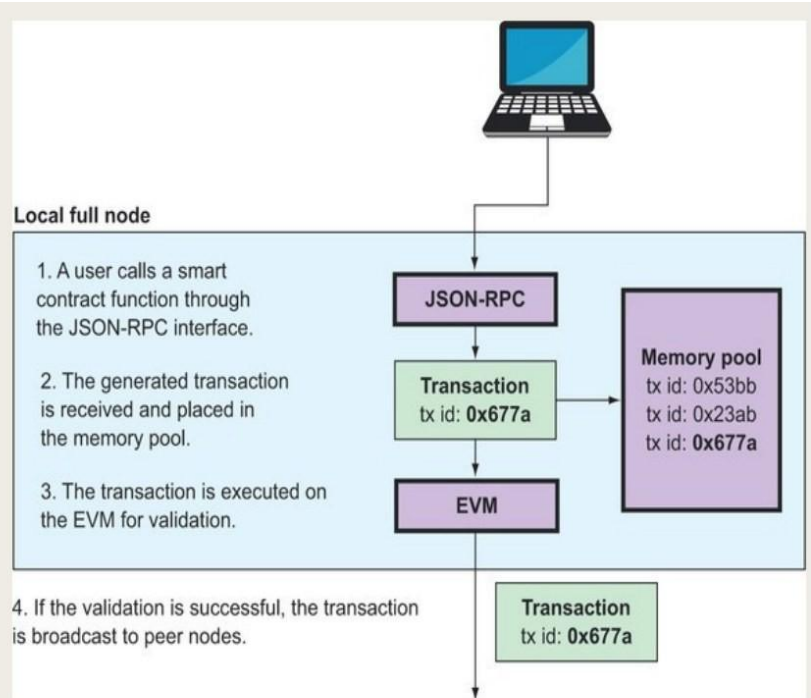


Slide image: Ethereum full node.

4.8 Ethereum transaction journey

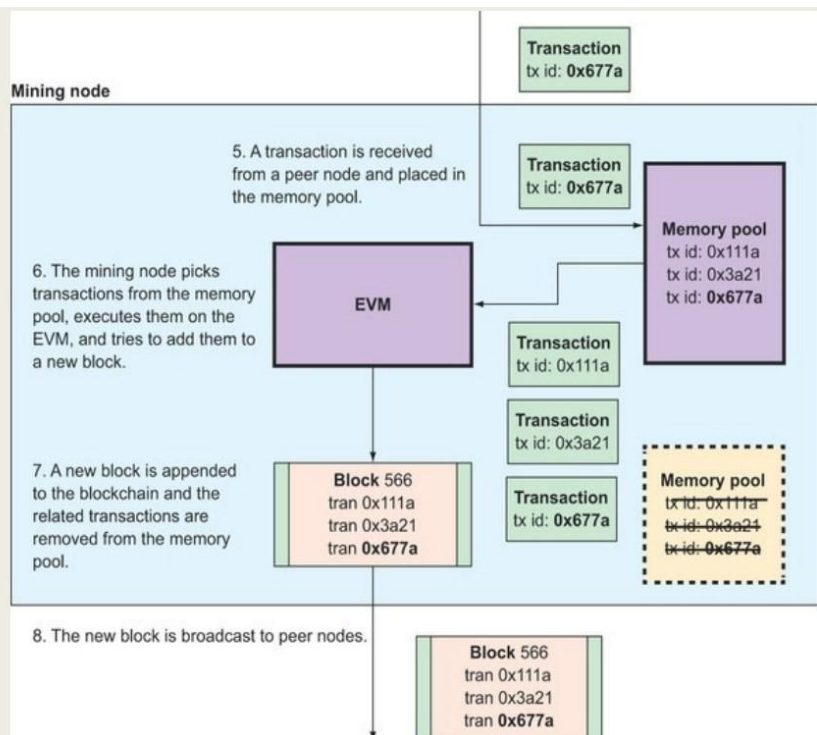
- 1 A voting DApp user invokes a voting function in a smart contract.
- 2 A voting transaction is generated through the JSON-RPC interface.
- 3 The node places the transaction in the memory pool.
- 4 The EVM executes it for validation.
- 5 If valid, the transaction is broadcast to peer nodes until it reaches a mining/validator node.
- 6 The mining/validator node stores it in its memory pool, later executes it with other transactions, and puts it in a block.
- 7 If the block is appended, the transaction is removed from the memory pool and the block is broadcast.
- 8 Full nodes receiving the new block re-execute transactions on the EVM for validation.
- 9 If validation succeeds, related mempool transactions are removed and the block propagates further.

The Journey of a Transaction



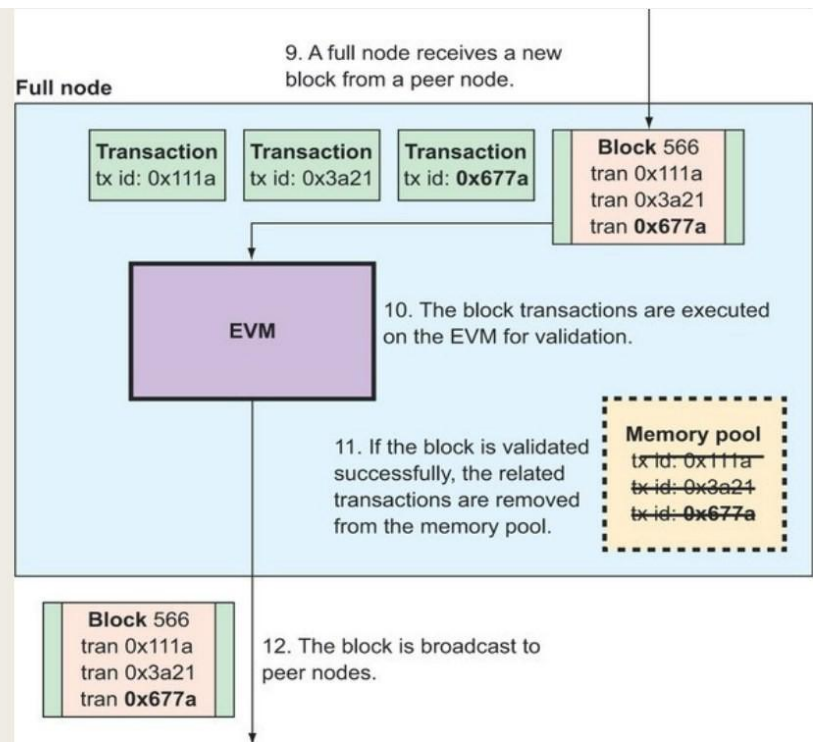
Slide image: transaction journey before mining/inclusion.

The Journey of a Transaction: The Mining Phase



Slide image: transaction journey during mining phase.

The Journey of a Transaction: When a full node receives a new block

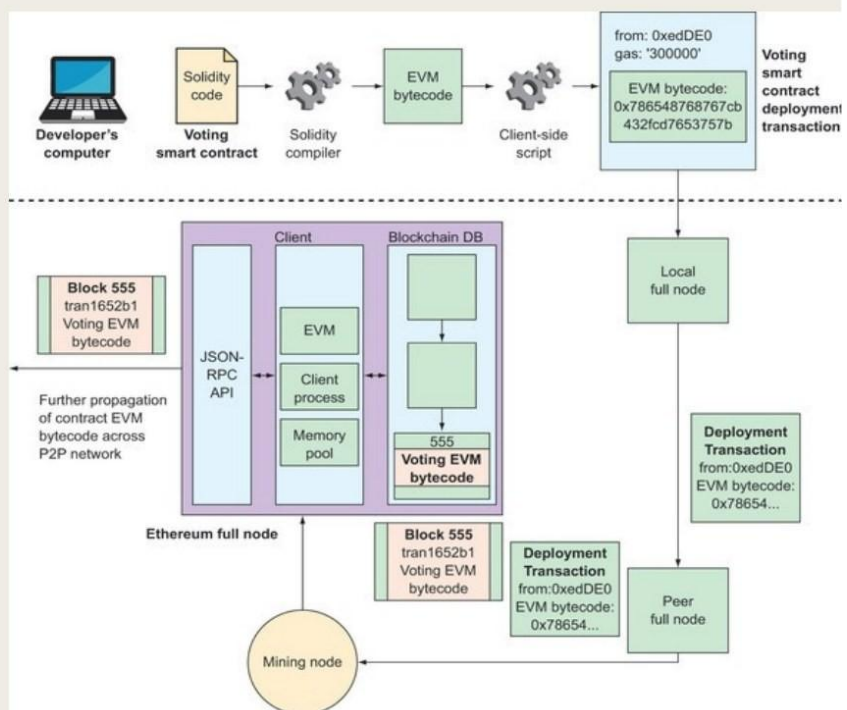


Slide image: full node receives and validates a new block.

4.9 Development view

The development view connects developer workflow to deployment: developer writes Solidity, compiler produces EVM bytecode and ABI, a deploy transaction is sent to the network, the contract receives an address, and users interact with the deployed contract through transactions/calls.

The Development View



Slide image: Ethereum development view.

4.10 Historical Ethereum proof-of-work: Ethash

Before The Merge, Ethereum used Ethash, a proof-of-work algorithm derived from Dagger-Hashimoto ideas. Ethash was memory-hard: mining depended on generating and frequently reading a large dataset called the DAG.

Ethash element	Meaning
Seed	Computed for each block by scanning previous block headers.
16 MB cache	Pseudorandom cache generated from the seed; light clients could store it.
DAG dataset	Large dataset initially about 1 GB and growing linearly over time.
Epoch update	Dataset updated every 30,000 blocks, roughly 125 hours in the slide.
Mining	Random slices of the dataset are read and hashed together.
Verification	Can be done with lower memory by regenerating needed pieces from the cache.

Purpose: make mining dependent on memory access, discouraging extreme ASIC advantage and reducing mining centralization pressure. Ethereum has now switched to proof-of-stake; the official Merge date is September 15, 2022, not Dec 2022.

5. Solidity by Example

5.1 What Solidity is

Solidity is a high-level, object-oriented smart contract language designed to target the Ethereum Virtual Machine. It is influenced by C++, Python, and JavaScript. The course document calls it a variant of JavaScript, but that is too narrow.

5.2 Counter example

The Counter contract demonstrates a persistent state variable and functions that read/write it.

```
pragma solidity ^0.8.26;
contract Counter {
    uint256 public count;
    function get() public view returns (uint256) { return count; }
    function inc() public { count += 1; }
    function dec() public { count -= 1; } // reverts if count is 0 in Solidity 0.8+
}
```

5.3 Primitive data types

Type	Meaning / range
bool	true or false; default false.
uint8	Unsigned integer from 0 to $2^8 - 1$.
uint16	Unsigned integer from 0 to $2^{16} - 1$.
uint256 / uint	Unsigned integer from 0 to $2^{256} - 1$; uint is alias for uint256.
int256 / int	Signed integer from -2^{255} to $2^{255} - 1$; int is alias for int256.
int128	Signed integer from -2^{127} to $2^{127} - 1$.
address	Ethereum address; default is 0x00.
bytes1	Fixed-size one-byte value.
bytes	Dynamic byte array; shorthand for byte[].

Solidity provides `type(int256).min` and `type(int256).max` for minimum and maximum int256 values.

5.4 Variable categories

Category	Where declared	Stored on-chain?
Local	Inside a function	No
State	At contract level	Yes
Global	Built-in blockchain context variables such as <code>block.timestamp</code> and <code>msg.sender</code>	Provided by environment

```
contract Variables {
    string public text = "Hello"; // state
    uint256 public num = 123;      // state
    function doSomething() public view {
        uint256 i = 456;          // local
        uint256 timestamp = block.timestamp;
        address sender = msg.sender;
    }
}
```

5.5 Constants and immutables

A constant is fixed at compile time. An immutable variable can be assigned once, usually in the constructor, and then cannot be modified.

```
address public constant MY_ADDRESS = 0x777788889999AaAaBbbCccddDdeeeEfffCcCc;
uint256 public constant MY_UINT = 123;

address public immutable myAddr;
uint256 public immutable myUInt;
constructor(uint256 _myUInt) {
    myAddr = msg.sender;
    myUInt = _myUInt;
}
```

5.6 Reading and writing state

Writing/updating a state variable requires a transaction. Reading a state variable using a view function can be free when called externally/off-chain because it does not change blockchain state.

```
contract SimpleStorage {
    uint256 public num;
    function set(uint256 _num) public { num = _num; }          // write, costs gas
    function get() public view returns (uint256) { return num; } // read
}
```

5.7 Ether and Wei

Unit	Value
1 wei	1
1 gwei	10 ⁹ wei
1 ether	10 ¹⁸ wei

5.8 Gas and gas limit

Transaction cost is gas spent multiplied by gas price. Gas measures computation/storage effort. Higher gas price/tip can increase priority for block inclusion. Unspent gas is refunded; gas spent by a failing transaction is not refunded.

```
transaction fee = gas spent * gas price
```

Term	Meaning
gas	Unit of computation.
gas spent	Total gas used by the transaction.
gas price	ETH willing to pay per gas.
gas limit	Max gas user allows for the transaction.
block gas limit	Max gas allowed in a block by the network.

Exam-important: if a transaction uses all provided gas, it fails, state changes are undone, and spent gas is not refunded.

5.9 Control flow and loops

Solidity supports if/else, ternary operator, for loops, while loops, continue, and break. Be careful with loops because infinite loops consume gas and revert.

```
function ternary(uint256 _x) public pure returns (uint256) {
    return _x < 10 ? 1 : 2;
}

for (uint256 i = 0; i < 10; i++) {
    if (i == 3) continue;
    if (i == 5) break;
}
```

5.10 Mappings

Mappings are key-value storage structures. They are not iterable. If a key was never set, the mapping returns the default value for the value type. delete resets a key to the default value.

```
mapping(address => uint256) public myMap;
mapping(address => mapping(uint256 => bool)) public nested;

function set(address _addr, uint256 _i) public { myMap[_addr] = _i; }
function remove(address _addr) public { delete myMap[_addr]; }
```

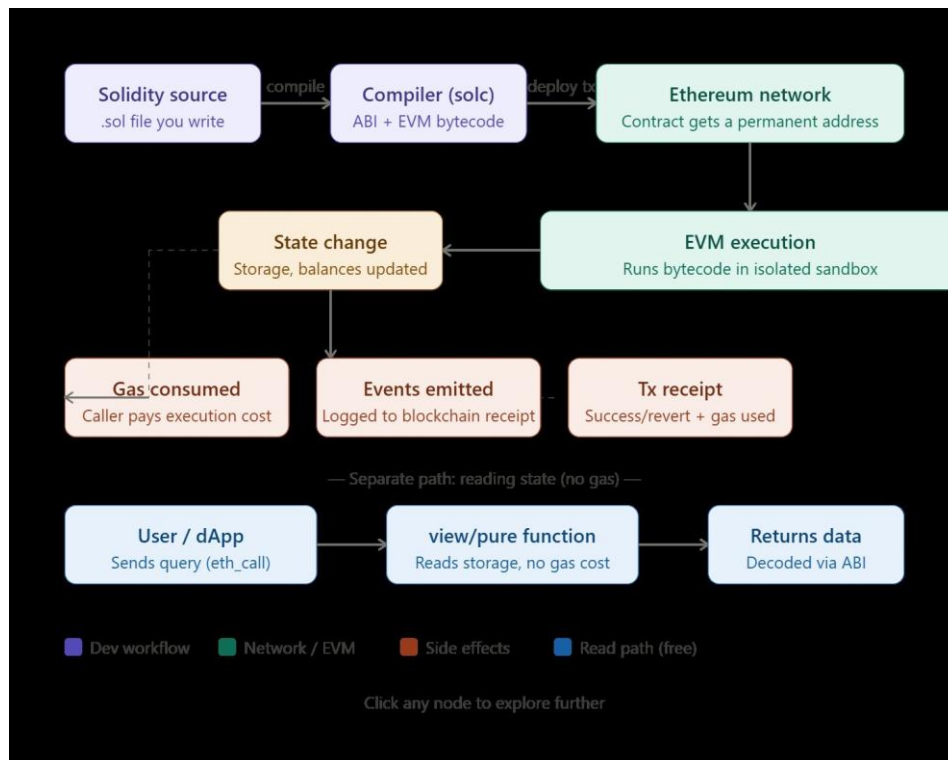
6. Ethereum Smart Contracts Practice Sheet

6.1 Smart contract definition

A smart contract is self-executing code stored on the Ethereum blockchain. Once deployed, it runs as written. It can automate payments, escrow, supply chain actions, insurance claims, and other if-condition-then-action workflows. The vending machine analogy is: if condition A is met, perform action B.

6.2 Architecture: how execution flows

The practice sheet architecture shows two paths: a state-changing transaction path and a read-only call path.



Smart contract architecture diagram from the practice sheet.

Step	Meaning
1. Solidity source	Developer writes a .sol file.
2. Compiler solc	Compiles to ABI + EVM bytecode.
3. Deploy transaction	Deploys the contract to the Ethereum network.
4. Permanent address	The contract receives an address.
5. EVM execution	Bytecode runs in an isolated sandbox.
6. State change	Storage/balances update if transaction succeeds.
7. Events and receipt	Events are logged; receipt records success/revert and gas used.
Read path	view/pure function via eth_call reads state and returns decoded data without gas cost to the caller off-chain.

6.3 SimpleStorage

SimpleStorage is the "Hello World" of smart contracts: it stores and retrieves a number on-chain.

```
contract SimpleStorage {
    uint256 private storedValue;
    event ValueChanged(address indexed by, uint256 newValue);
    function set(uint256 _value) public {
        storedValue = _value;
        emit ValueChanged(msg.sender, _value);
    }
    function get() public view returns (uint256) { return storedValue; }
}
```

- uint256: unsigned 256-bit integer.
- public: accessible from outside the contract.
- view: read-only; no state change.
- emit: logs an event to the transaction receipt.
- msg.sender: address that called the function.
- set(42) costs gas because it writes state; get() returns 42 with no gas cost when called externally/off-chain.

6.4 ERC-20 token contract

ERC-20 is the common standard for fungible tokens. The example uses OpenZeppelin ERC20 and Ownable, mints initial supply to the deployer, and restricts additional minting to the owner with a max supply check.

```
contract MyToken is ERC20, Ownable {
    uint256 public constant MAX_SUPPLY = 1_000_000 * 10**18;
    constructor(address initialOwner) ERC20("MyToken", "MTK") Ownable(initialOwner) {
        _mint(initialOwner, 100_000 * 10**18);
    }
    function mint(address to, uint256 amount) public onlyOwner {
        require(totalSupply() + amount <= MAX_SUPPLY, "Exceeds max supply");
        _mint(to, amount);
    }
}
```

6.5 Voting example

The VotingSystem contract stores vote counts for candidate A and candidate B. voteCandidateA and voteCandidateB increment counters; getVotes returns both counts.

```
contract VotingSystem {
    uint public candidateA;
    uint public candidateB;
    function voteCandidateA() public { candidateA += 1; }
    function voteCandidateB() public { candidateB += 1; }
    function getVotes() public view returns (uint, uint) { return (candidateA, candidateB); }
}
```

6.6 Simple token transfer

The SimpleTokenTransfer contract demonstrates balances using mapping(address => uint). mint increases the caller's balance; transfer checks the sender has enough balance, subtracts from sender, and adds to recipient.

```
contract SimpleTokenTransfer {
    mapping(address => uint) public balances;
    function mint(uint amount) public { balances[msg.sender] += amount; }
    function transfer(address recipient, uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        balances[recipient] += amount;
    }
}
```

6.7 SimpleBank

The SimpleBank contract uses payable deposit to receive ETH, stores balances by address, and withdraws by transferring ETH to msg.sender after checking sufficient funds.

```
contract SimpleBank {
    mapping(address => uint) balances;
    function deposit() public payable { balances[msg.sender] += msg.value; }
    function withdraw(uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient funds");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }
    function checkBalance() public view returns (uint) { return balances[msg.sender]; }
}
```

Security note: the practice example is educational. Real money contracts need stronger patterns and auditing, especially around withdrawals and external calls.

6.8 Vending machine

The VendingMachine contract sets the owner at deployment, starts with 100 cupcakes in the contract address, and lets users purchase cupcakes for 1 ETH each if there is enough stock.

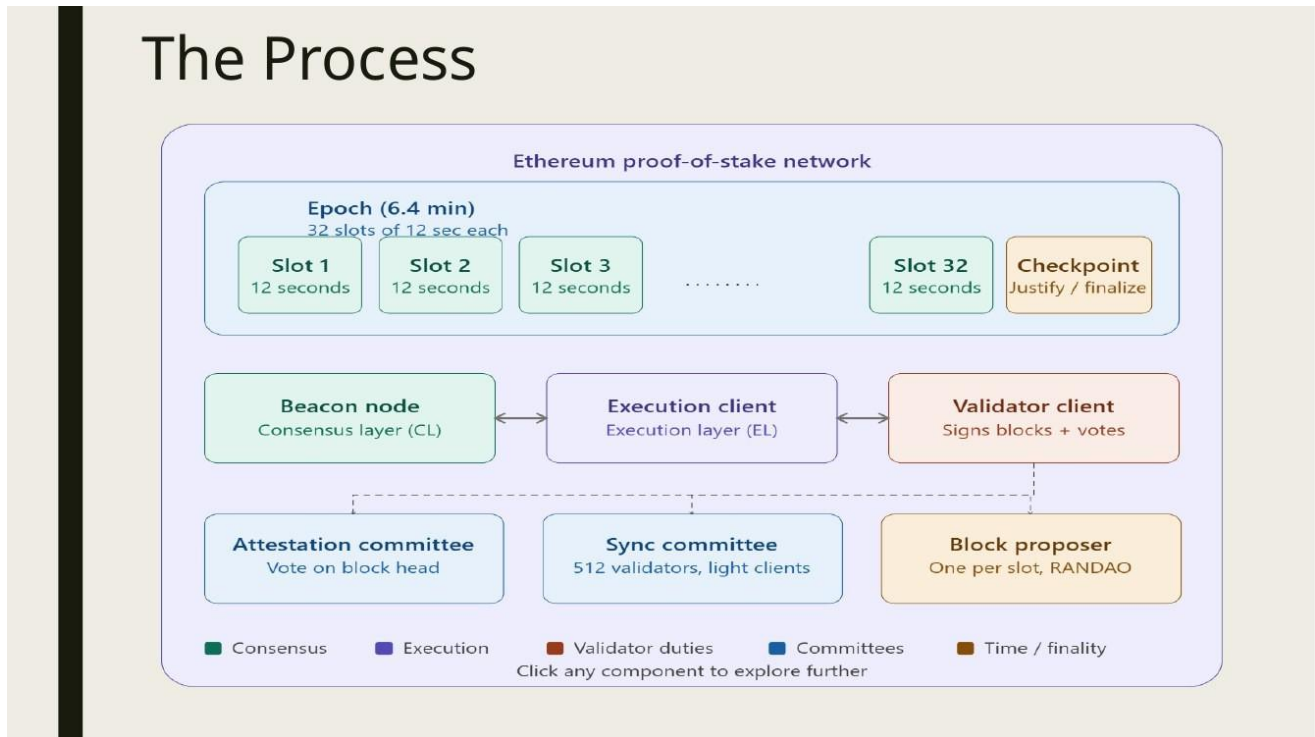
```
contract VendingMachine {
    address public owner;
    mapping(address => uint) public cupcakeBalances;
    constructor() {
        owner = msg.sender;
        cupcakeBalances[address(this)] = 100;
    }
    function purchase(uint amount) public payable {
        require(msg.value >= amount * 1 ether, "Pay 1 ETH per cupcake");
        require(cupcakeBalances[address(this)] >= amount, "Not enough stock");
        cupcakeBalances[address(this)] -= amount;
        cupcakeBalances[msg.sender] += amount;
    }
}
```

7. Ethereum Proof-of-Stake

7.1 Stake, not compute

Proof-of-work makes cheating expensive by requiring energy and computation. Proof-of-stake makes cheating expensive by requiring validators to lock ETH as collateral. Ethereum validators stake 32 ETH. If they act dishonestly, some or all of that stake can be destroyed.

Exam-important: PoS replaces energy-burning competition with capital-at-risk security.



Slide image: Ethereum PoS process.

7.2 Node/client components

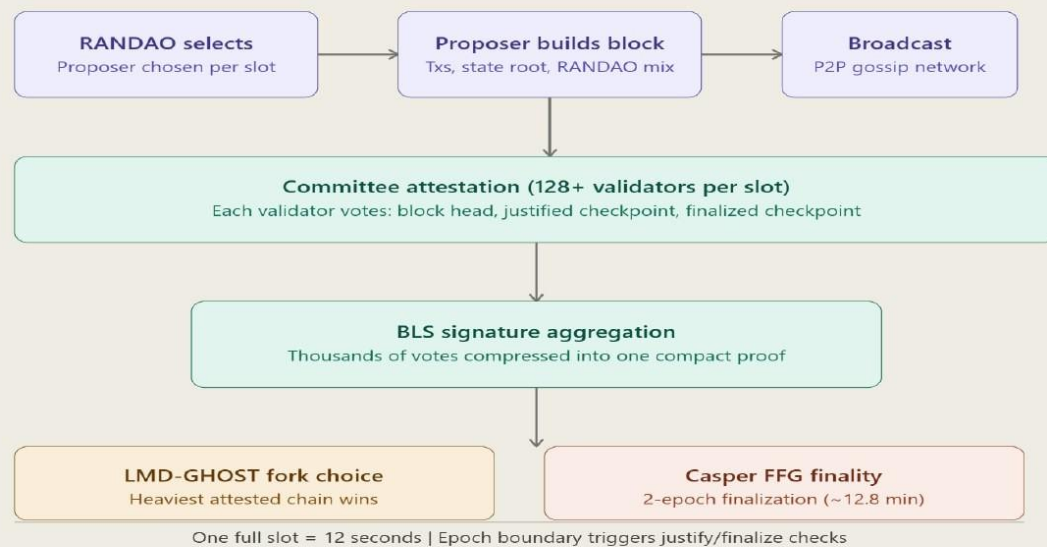
Component	Role
Beacon node / consensus client	Runs Gasper, maintains beacon chain, tracks validator registry, runs RANDAO, fork choice, and finality. Examples: Prysm, Lighthouse, Teku, Nimbus, Lodestar.
Execution client	Runs EVM, maintains world state, validates transactions, builds execution payloads, manages mempool. Examples: Geth, Nethermind, Besu, Erigon.
Validator client	Holds signing keys; proposes blocks when selected and casts attestations. Must not sign conflicting messages.

7.3 Gasper consensus

Ethereum's consensus combines LMD-GHOST and Casper FFG.

Gasper = LMD-GHOST + Casper FFG	
Protocol	Role
LMD-GHOST	Fork-choice rule. It answers: given competing chain tips, which one should validators build on? Only each validator's latest message counts.
Casper FFG	Finality gadget. Validators vote on checkpoint pairs; if enough stake supports them, checkpoints become justified/finalized.

The Consensus Protocol



Slide image: Gasper consensus flow.

7.4 Slots and epochs

Term	Meaning
Slot	12 seconds. One validator is randomly selected as block proposer per slot.
Epoch	32 slots, about 6.4 minutes.
Committee	Random subset of validators selected to attest; all active validators attest once per epoch but not every slot.
Checkpoint	First block in each epoch; used by Casper FFG for finality.

7.5 Transaction execution in Ethereum PoS

- 1 User creates and signs a transaction with a private key through a wallet/library.
- 2 User defines gas and tip; the tip goes to the validator/proposer and the base fee is burned.
- 3 Transaction goes to an execution client through JSON-RPC.
- 4 Execution client verifies signature, sender balance, and validity.
- 5 Valid transaction enters the local mempool and is gossiped to other nodes.
- 6 The selected proposer for the current slot uses RANDAO-based selection.
- 7 Execution client bundles mempool transactions into an execution payload and executes locally.
- 8 Consensus client wraps the execution payload into a beacon block with rewards/penalties/Slashings/attestations metadata.
- 9 Other nodes receive the beacon block, re-execute the transactions, and attest if valid.
- 10 Finality occurs through supermajority checkpoint links between epochs.

7.6 Finality

A transaction has finality when it is part of a block that cannot change without large amounts of ETH being burned/slashed. Validators vote for checkpoint pairs. If votes represent at least two-thirds of total staked ETH, the target checkpoint becomes justified and the previous justified checkpoint becomes finalized.

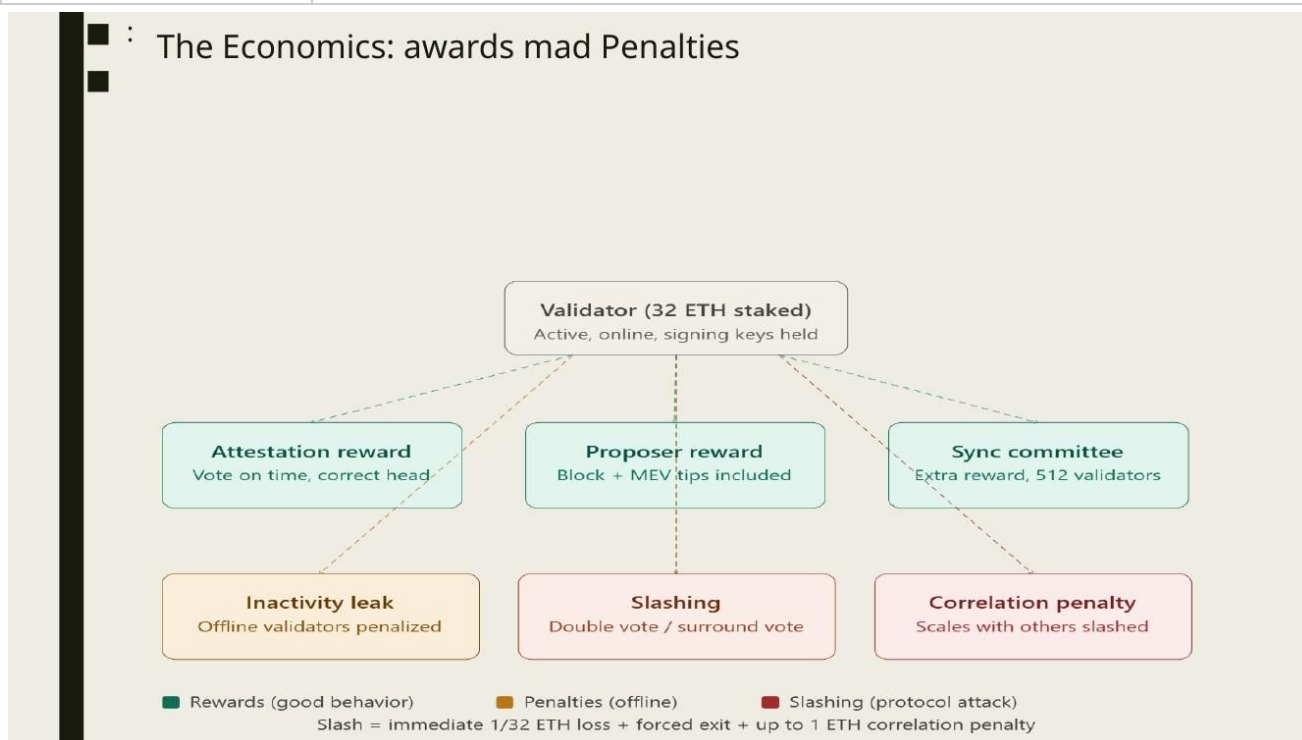
Exam-important: finality is not just "many confirmations." In Ethereum PoS, it is an economic/protocol condition involving checkpoint votes and potential slashing.

7.7 Committee, sync committee, and BLS aggregation

The diagrams show one proposer per slot, attestation committees, a sync committee of 512 validators for light-client support, and BLS signature aggregation. BLS aggregation compresses many validator signatures/votes into compact proofs so the network can process thousands of attestations efficiently.

7.8 Rewards and penalties

Category	Examples
Rewards	Attestation rewards, proposer rewards, sync committee rewards, MEV/tips.
Penalties	Offline/inactivity penalties, inactivity leak during finality failures, slashing for protocol attacks.
Correlation penalty	Penalty increases if many validators are slashed together, suggesting coordinated attack or shared infrastructure failure.



Slide image: economics - rewards and penalties.

7.9 Slashable offenses

Offense	Meaning
Equivocation / double voting	Validator signs conflicting blocks/votes for the same slot.
Surround voting	Validator casts two Casper FFG votes where one vote surrounds the other.

Slashable behavior is detected automatically through slashing proofs submitted by other validators. The submitter can receive a portion of the slashed stake as reward.

7.10 RANDAO randomness

Validator/proposer selection must be random and hard to predict. Ethereum uses RANDAO: proposers mix BLS-signature-derived entropy into a running accumulator. The result seeds validator shuffling and committee/slot assignments. A single malicious proposer can only weakly bias randomness by refusing to propose, which costs them rewards.

8. Solana: Proof of History and Fast Transaction Processing

8.1 Third-generation blockchain context

The Solana deck frames newer blockchain systems as third/fourth generation designs that address older limitations using innovations such as sharding, zero-knowledge proofs, hybrid consensus, faster/greener operation, and better interoperability. Solana is presented as a high-throughput chain using Proof of History plus Proof of Stake.

8.2 What Solana is

Solana is a high-performance public blockchain supporting smart contracts, decentralized applications, and digital assets. It prioritizes speed, throughput, and cost to provide a Web2-like user experience. The slides compare Solana's theoretical 65,000 TPS to Ethereum's lower base-layer throughput.

8.3 Story

Solana's story starts in 2017 with Anatoly Yakovenko and the Proof of History concept. Solana Labs was established in 2017, testnets followed, and mainnet beta launched in 2020 after public/private funding rounds.

8.4 Solana Proof of Stake

SOL holders can delegate their stake to validators. Validators create new blocks, secure the network, earn SOL rewards, and share rewards with delegators. The slides emphasize high validator hardware requirements: about 256 GB RAM, a 12-core CPU, and over 1 TB storage.

The slide also notes a "superminority": 21 validators controlling more than 33% of total stake. This matters because one-third stake thresholds are important for BFT-style liveness/security analysis.

How Does it Work?

In what follows, we describe how Solana works (since its launch in 2020 –the situation will drastically change after the upcoming Alpenglow upgrade)

Proof-of-Stake (PoS)

- Solana relies on a **PoS version** where regular SOL holders delegate their staked coins to validators, who form the network's backbone.
- **Validator nodes** ensure Solana's functionality and security, creating new blocks with transactions. They earn SOL rewards based on their share of staked coins. Part of the rewards is shared with delegators based on their staking contribution.
- Validators on Solana use high-performance hardware, with requirements much higher than those on Ethereum. For example, the minimum requirements to **run a Solana node** are 256 GB of RAM (versus 8 GB on Ethereum), a 12-core CPU, and over 1 TB of storage space.
- As of this writing, there are **1,240 validators**, 21 of which form the so-called superminority, as they control over 33% of the total stake. Some of the biggest validators include Helius, Binance, Coinbase, Ledger, and SOL Community.
- Solana validators are dispersed across all continents, though with a higher concentration in Europe and North America.

7

Important diagram: Solana PoS, validator hardware, and superminority note.

8.5 Proof of History

Proof of History is Solana's cryptographic clock. Transactions/events are placed into a sequence of SHA-256 hashes, giving them a verifiable order without validators constantly communicating to decide which event came first.

Exam-important: PoH is not consensus by itself. It provides ordering/time. Solana still uses stake-weighted validator voting/Tower BFT-style consensus for agreement.

Proof of History

What sets Solana apart is its PoH mechanism, a cryptographic clock that timestamps and verifies the sequence of all network events.

- PoH enables validators to handle transactions in parallel, boosting throughput and reducing congestion. By timestamping each transaction before reaching a block, PoH streamlines the validation process.
- The PoH algorithm is closely associated with Solana's Tower BFT (Byzantine Fault Tolerance) system, a consensus mechanism that helps validators agree even if some nodes fail or behave maliciously.
- Thanks to PoH, validators know the exact order of transactions without having to communicate with each other constantly. This way, Tower BFT reduces the communication requirements typical of other chains.
- Validators vote on the validity of each new block using Tower BFT. The blocks are added if at least 66% of validators approve them.

8

Important diagram: Proof of History + Tower BFT relationship.

How to read it: PoH gives each event a cryptographic place in time; Tower BFT/PoS voting decides agreement/finality.

8.6 Tower BFT

The slides describe Tower BFT as a Byzantine Fault Tolerance consensus system that helps validators agree even if some nodes fail or behave maliciously. Validators vote on new blocks, and blocks are added if at least about 66% of validators approve them, according to the slide.

8.7 Proof of History transaction process

- 1 User submits a trade transaction to exchange Token A for Token B.
- 2 Transaction enters the PoH sequence.
- 3 It is hashed into Solana's ongoing SHA-256 chain and receives a verifiable timestamp.
- 4 The timestamp proves relative order without an external clock.
- 5 Validators can pre-order transactions without wasting time on ordering communication.
- 6 PoS + Tower BFT handles final agreement and confirmation.
- 7 Trade finalizes and settles faster than traditional blockchains requiring more coordination.

8.8 Firedancer and Alpenglowl

The slides mention two major upgrades. Firedancer is an independent validator client developed by Jump Crypto, intended to improve performance and client diversity. Alpenglowl is a proposed consensus overhaul intended to sharply reduce finality latency; the Solana forum proposal describes roughly 100-150 ms latency targets.

Treat slide performance numbers as time-sensitive. For exams, focus on what each upgrade is meant to do: Firedancer = validator client/performance/client diversity; Alpenglowl = consensus/finality redesign.

8.9 Other third-generation chains mentioned

Chain	Core idea from deck	Difference emphasized
Cardano	Ouroboros PoS, research/peer-reviewed style, Hydra L2, governance.	More academic/research-driven.
Avalanche	X/P/C chains, quick finality, subnets.	Custom subnets and fast finality.
Cosmos	Tendermint BFT, IBC, sovereign chains.	Interoperable network of independent chains.

Third Generation:		
Blockchain	Key Features	How It Differs from Ethereum
Cardano	- Proof-of-Stake (Ouroboros) - Peer-reviewed research - Hydra (Layer 2) for 1M+ TPS - On-chain governance	- More academic & secure than Ethereum - Slower development but no major outages - Native multi-asset support (vs. ERC-20)
Avalanche	- 3 chains: X (exchange), P (platform), C (contract) - Sub-second finality - 4,500+ TPS - Avalanche Warp Messaging (cross-subnet)	- Faster finality than Ethereum - Custom subnets = private blockchains with shared security - No gas wars
Cosmos	- Tendermint BFT consensus - IBC (Inter-Blockchain Communication) - 100+ sovereign chains connected	- Not a single chain — a network of chains - Sovereign interoperability (vs. Ethereum bridges) - Used by Binance Chain, Terra (pre-collapse)

11

Important comparison slide: third-generation blockchains mentioned in the Solana deck.

9. Polkadot: Blockchain of Blockchains

9.1 In a nutshell

Polkadot is a decentralized, scalable, interoperable blockchain network built around a central Relay Chain and connected parachains. It supports custom blockchains while maintaining shared security and interoperability. DOT is used for governance, staking, and network security/resource access. The deck also emphasizes forkless upgrades, democratic governance, and Nominated Proof of Stake.

9.2 Story

The deck says the Web3 Foundation, co-founded by Gavin Wood, held a 2017 ICO raising about \$144 million. About \$90 million was frozen due to a Parity wallet vulnerability, but the project continued. Polkadot's genesis block launched in 2020. Polkadot v1 focused on the main architecture; Polkadot 2.0 represents a more mature upgraded stage.

9.3 Main components

Component	Meaning
Relay Chain	Main chain coordinating the system, interoperability, consensus, and shared security.
Parachains	Independent specialized blockchains connected to the Relay Chain; can have their own tokens and logic.
Bridges	Special parachains/connections allowing communication with external blockchains such as Ethereum.

Main Components of Polkadot

Polkadot works with three specific components that make it unique: the relay chain, parachains, and bridges.

- Relay Chain
 - This is Polkadot's main blockchain that coordinates the system, handles interoperability, and provides shared security.
 - Polkadot uses a consensus mechanism called nominated proof-of-stake (NPoS)
 - NPoS ensures the integrity of the blockchain. It also provides a decentralized governance system and validates transactions.
- Parachains
 - These are independent blockchains connected to the relay chain, and each of them can have their own tokens.
 - Polkadot's parachains provide customizability and scalability.
 - For example, a parachain can specialize in identity verification while another one specializes in real estate transactions.
- Bridges
 - Bridges are special parachains that allow Polkadot to connect and communicate with other blockchains like Ethereum.
 - As a result, they facilitate interoperability and expand the network's capabilities, like cross-chain communication and asset transfer.

5

Important diagram/text slide: Polkadot Relay Chain, parachains, and bridges.

How to read it: Relay Chain coordinates and secures; parachains are specialized parallel chains; bridges connect to external chains.

9.4 DOT token functions

Function	Meaning
Governance	DOT holders vote on fees, auctions, parachain changes, and protocol changes.
Staking	DOT is locked to support network operations, validator/nominator selection, and security.

Function	Meaning
Bonding / parachain auctions	Projects bond DOT to win a parachain slot for a fixed period, usually up to two years in the slide.

9.5 Scalability and finality

Polkadot improves scalability by processing transactions in parallel across multiple parachains. Each parachain operates independently instead of forcing all transactions through one global sequential chain. The deck also says Polkadot uses advanced finality mechanisms to secure the Relay Chain.

Polkadot's official technical documentation describes a hybrid consensus model: BABE for block production and GRANDPA for provable finality. This is more detailed than the slide, but useful for correctness.

9.6 Benefits and advantages

Benefit	Meaning
Interoperability	Different blockchains can share information/assets.
Scalability	Multiple parachains process transactions in parallel.
Shared security	Parachains benefit from Relay Chain security.
Customizability	Developers can tailor blockchains for specific use cases.
Ecosystem development	Encourages collaborative development and innovation.
Forkless upgrades	Network can upgrade without disruptive hard forks.
Democratic governance	Communities can govern and evolve the network.

Slide 8 is mislabeled "Proof of History". Its actual content is Polkadot benefits, not Solana PoH.

9.7 Nominated Proof of Stake

Role	Function
Validators	Secure the Relay Chain, validate parachain transactions, participate in consensus, and produce blocks.
Nominators	Stake DOT behind trustworthy validators, supporting network security.
Collators	Manage parachain transactions and submit valid candidates to Relay Chain validators.
Fishermen	Monitor and report misconduct.

Exam-important: Validators do core validation; nominators back validators with stake; collators handle parachain transaction candidates; fishermen report bad behavior.

Nominated Proof of Stake (Npos)

- Polkadot utilizes a nominated proof-of-stake (NPoS) consensus mechanism involving validators and nominators.
- Validators secure the Relay Chain by staking DOT, validating transactions on the parachains, participating in consensus with other validators and producing blocks on the Relay Chain.
- Nominators, on the other hand, contribute to the network's security by staking their DOT to back trustworthy validators. This system enhances the network's security while enabling cross-chain communication and interoperability.
- Polkadot also includes two specialized roles: Collators, who manage and submit valid parachain transactions to Relay Chain validators, and Fishermen, who monitor and report network misconduct.
- These roles require more technical skill than nominators but less commitment than full validators, playing a vital role in maintaining the network.

11

Important slide: Nominated Proof of Stake roles.

10. Key terms

Term	Definition
Blockchain	Append-only distributed ledger secured by cryptography and consensus.
P2P network	Network where peers share data without a central server.
Block	Container of transactions and metadata.
Block header	Metadata used for linking, mining, and validation.
Hash	Fixed-size digest of data; small input changes cause major output changes.
Previous block hash	Hash pointer linking a block to its predecessor.
Merkle root	Single root hash summarizing many data items.
Digital signature	Cryptographic proof that a private-key holder authorized data.
Double spending	Attempt to spend the same digital value twice.
Proof-of-work	Consensus mechanism based on computational hash search.
Target	Maximum acceptable hash value in PoW.
Difficulty	Measure of how hard it is to find a valid PoW hash.
Nonce	Value miners vary to search for valid hashes.
UTXO	Unspent Transaction Output; Bitcoin spendable unit.
EOA	Externally Owned Account; Ethereum account controlled by a private key.
Contract account	Ethereum account controlled by code.
EVM	Ethereum Virtual Machine; executes smart contract bytecode.
Gas	Unit measuring computation/storage effort in Ethereum.
Solidity	High-level smart contract language for EVM.
ABI	Interface description used by apps to call contract bytecode.
Wei	Smallest ETH denomination.
Validator	PoS participant that proposes and attests blocks.
Attestation	Validator vote supporting a block/checkpoint.
Finality	Condition where reverting a block would require severe economic/protocol penalties.
LMD-GHOST	Ethereum fork-choice rule.
Casper FFG	Ethereum finality gadget.
RANDAO	Ethereum decentralized randomness mechanism.
Proof of History	Solana cryptographic clock/order mechanism.
Tower BFT	Solana BFT-style validator voting consensus described in the slides.
Relay Chain	Polkadot central chain providing coordination and shared security.
Parachain	Specialized chain connected to Polkadot Relay Chain.
Bridge	Connection to external blockchains.
NPoS	Polkadot nominated proof-of-stake mechanism.

11. Quick revision cheat sheet

Bitcoin

- Bitcoin = P2P electronic cash.
- Core problem solved = double spending without trusted third party.
- Block header = version, previous hash, Merkle root, timestamp, difficulty bits, nonce.
- PoW challenge = find header hash below target.
- Lower target = higher difficulty.
- Difficulty adjusts every 2,016 blocks toward a 10-minute average block interval.
- Bitcoin uses UTXO model: inputs, outputs, change, fee.
- Wallet manages keys and builds/signed transactions; it does not literally store coins.
- Accepted chain = chain with most cumulative proof-of-work.

Ethereum

- Ethereum = programmable blockchain for smart contracts and DApps.
- EOA = private-key account; contract account = code-controlled account.
- Account state = nonce, balance, storageRoot, codeHash.
- EVM executes smart contract bytecode.
- State writes cost gas; off-chain view calls can be free.
- Historical PoW algorithm = Ethash, memory-hard with cache + DAG.
- Ethereum moved to PoS on September 15, 2022.
- PoS validator stake = 32 ETH.
- Gasper = LMD-GHOST + Casper FFG.
- Slot = 12 seconds; epoch = 32 slots.

Solidity and smart contracts

- Solidity variables: local, state, global.
- uint = uint256; int = int256.
- msg.sender = caller; block.timestamp = current block timestamp.
- payable functions can receive ETH.
- require() checks a condition and reverts on failure.
- mapping(key => value) is not iterable and returns default values for unset keys.
- Events are logs stored in transaction receipts.
- ERC-20 = standard for fungible tokens.

Solana

- Goal = high throughput, low cost, low latency.
- Uses PoS plus Proof of History.
- PoH = cryptographic clock/order sequence, not full consensus alone.
- Tower BFT = validator agreement/voting mechanism in the slides.
- Firedancer = independent validator client/performance/client diversity.

- Alpenglow = proposed consensus/finality redesign.

Polkadot

- Polkadot = blockchain of blockchains.
- Relay Chain = central coordination/shared security.
- Parachains = specialized parallel chains.
- Bridges = communication with external blockchains.
- DOT = governance, staking, bonding/resource access.
- NPoS roles = validators, nominators, collators, fishermen.
- Key advantages = interoperability, scalability, shared security, customizability, forkless upgrades.

12. Possible exam questions with answers

1. What was Bitcoin designed to solve?

Peer-to-peer electronic cash without a financial institution, especially the double-spending problem.

2. Why does changing a transaction change the block?

Transactions are summarized in the Merkle root. Changing one transaction changes hashes up the tree, changes the Merkle root, and changes the block header hash.

3. What is the PoW target?

The maximum acceptable hash value. A valid block hash must be numerically below the target.

4. Explain target vs difficulty.

They are inversely related. Lower target means fewer acceptable hashes and higher difficulty.

5. Why does difficulty adjust every 2,016 blocks?

To keep the average block interval near 10 minutes by comparing actual time to expected 20,160 minutes.

6. What is the UTXO model?

Bitcoin tracks unspent transaction outputs. Transactions consume previous outputs as inputs and create new outputs.

7. EOA vs contract account?

EOA is controlled by a private key and initiates transactions. Contract account is controlled by code and runs when called.

8. What are Ethereum account state fields?

nonce, balance, storageRoot, codeHash.

9. What does the EVM do?

Executes smart contract bytecode and computes state changes.

10. Why does state writing cost gas?

It changes blockchain state and must be validated/executed by the network.

11. What is Ethash?

Ethereum's historical memory-hard PoW algorithm using seed, cache, DAG, and random dataset access.

12. What is Gasper?

Ethereum PoS consensus composed of LMD-GHOST fork choice and Casper FFG finality.

13. What is finality in Ethereum PoS?

A block/checkpoint is finalized when reverting it would require severe slashing/economic loss after two-thirds stake checkpoint voting.

14. Name two slashable Ethereum behaviors.

Double voting/equivocation and surround voting.

15. What is RANDAO used for?

Decentralized randomness for validator/proposer and committee selection.

16. What is Proof of History?

Solana's cryptographic clock that gives events verifiable order through a hash sequence.

17. Is PoH consensus by itself?

No. It gives timing/order; validator voting/PoS/Tower BFT handles agreement.

18. What is Polkadot's Relay Chain?

The central chain that coordinates the system, shared security, interoperability, and consensus.

19. What are parachains?

Independent specialized blockchains connected to the Relay Chain, processing transactions in parallel.

20. What are the NPoS roles?

Validators, nominators, collators, and fishermen.

13. Highest-priority focus topics

- 1 Bitcoin block structure and header fields: previous hash, Merkle root, difficulty bits, nonce.
- 2 Proof-of-work mechanics: target, difficulty, nonce search, difficulty adjustment.
- 3 Transaction lifecycle: wallet signing, broadcasting, mempool, mining/validation, confirmation.
- 4 Bitcoin UTXO model: inputs, outputs, change, fees.
- 5 Ethereum account model: EOA vs contract account; account state fields.
- 6 EVM and gas: why gas exists, fee formula, gas limits, revert behavior.
- 7 Solidity fundamentals: variables, mappings, view, payable, require, events, constructor, ETH units.
- 8 Smart contract practice examples: SimpleStorage, ERC-20, Voting, Token Transfer, Bank, Vending Machine.
- 9 Ethereum PoS: 32 ETH validators, slots/epochs, attestations, finality, Gasper, LMD-GHOST, Casper FFG.
- 10 Slashing and validator economics: double voting, surround voting, inactivity penalties, correlation penalties.
- 11 Solana: PoH vs PoS/Tower BFT; Firedancer and Alpenglow at a high level.
- 12 Polkadot: Relay Chain, parachains, bridges, DOT functions, NPoS roles, interoperability.

14. Final coverage checklist

This exam-ready version is meant to be studied directly. The huge full-slide appendix from the earlier PDF was removed so you do not waste time jumping around. All important diagrams have been placed inside the chapter where they belong.

Source	Covered in this PDF
_021 Blockchain Overview.pptx - 26 slides	Yes: origin, P2P, block structure/header, double spending, transactions, PoW, architecture, transaction lifecycle, mining, consensus, features.
_022 Bitcoin Proof of Work Algorithm.pptx - 13 slides	Yes: target formula, hashing header+nonce, toy SHA-256 nonce example, leading-zero/difficulty, time doubling, difficulty adjustment.
_023 How Bitcoin Works.pptx - 11 slides	Yes: network overview, transaction meaning, UTXO inputs/outputs/change/fee, wallet role, real transaction, mining/confirmation diagrams.
_024 Ethereum How It Works -1.pptx - 32 slides	Yes: overview, story, accounts, account state, state tree, Merkle tree, full node, transaction journey, development view, historical Ethash.
_025 Solidity By Example1.docx	Yes: Counter, primitive types, variables, constants/immutable, read/write state, Ether units, gas, control flow, loops, mappings.
_026 Ethereum Smart Contracts Practice Sheet.docx	Yes: smart contract architecture, SimpleStorage, ERC-20, Voting, Token Transfer, Bank, Vending Machine examples.
_027 Ethereum Proof of Stake.pptx - 19 slides	Yes: stake not compute, validator setup, clients/nodes, Gasper, LMD-GHOST, Casper FFG, slots/epochs, transaction execution, rewards/penalties, slashing, RANDAO.
_028 Solana Proof of History.pptx - 11 slides	Yes: Solana overview, third-generation context, story, PoS, validator hardware, supermajority, PoH, Tower BFT, transaction process, Firedancer/Alpenglow, comparisons.
_029 Polkadot.pptx - 11 slides	Yes: story, Relay Chain, parachains, bridges, DOT functions, auctions/bonding, scalability/finality, features, NPoS roles, mislabeled PoH slide note.

Study order for highest score: (1) chapter notes and diagrams, (2) Solidity examples, (3) cheat sheet, (4) exam questions without looking at answers, (5) final focus topics.